ElectriSys · Sprint 3

Documentación de la base de datos Oracle

Modelo entidad-relación, diccionario de datos y normalización (3FN / FNBC)

Cliente: Electritorres (repuestos automotrices y agrícolas)

Autores: Cristian Camilo Pesillo Diazgranados · Andrés Santiago Torres Garzón

Asignatura: Ingeniería de Software

Estado de los scripts SQL

Los scripts nuevos de este sprint (07, 09 y 15) **no se han ejecutado contra una instancia Oracle real**: se escribieron para ser re-ejecutables y deben probarse primero en una copia de la base. Ver el LEEME del paquete oracle_sprint3.

1. Alcance y estado de la base de datos

La base de datos de ElectriSys corre en **Oracle 23ai** y se crea con los scripts de la carpeta `oracle/` mediante `cargar_esquema.sh`, en este orden: tablas (02), índices (03), triggers (04), paquetes PL/SQL (06), vistas (07), roles y usuarios (08, 09), pedidos (10) y las tablas del asistente (12–14). Este documento describe el estado resultante después de aplicar el script 15 (fase 1 de normalización).

Elemento	Cantidad / detalle
Tablas	25 (21 del núcleo del negocio y del asistente + MARCAS, MODELOS, CONSULTA_ENTIDADES y CONSULTA_RECOMENDACIONES del script 15)
Índices	17 en el script 03, más los de unicidad y claves foráneas
Triggers	9 en el script 04, más `trg_vehiculos_modelo_id` del script 15
Paquetes PL/SQL	pkg_ventas, pkg_reportes
Vistas	4 originales + `vw_totales_documento` y `vw_vehiculos`
Roles	rol_electrisys_admin, rol_electrisys_vendedor, rol_electrisys_dueno

Nota sobre el conteo. El plan de trabajo hablaba de 21 tablas en 3FN. El número real es 25 porque se agregaron las tablas del asistente de lenguaje natural (nlp_intenciones, nlp_frases, posibles_errores, posibles_errores_avisos) y las 4 tablas nuevas de la normalización.

2. Modelo entidad-relación

El modelo se divide en tres diagramas para que sea legible. Cada caja muestra solo las claves (PK y FK); las demás columnas están en el diccionario de datos (sección 3).

2.1 Módulo comercial

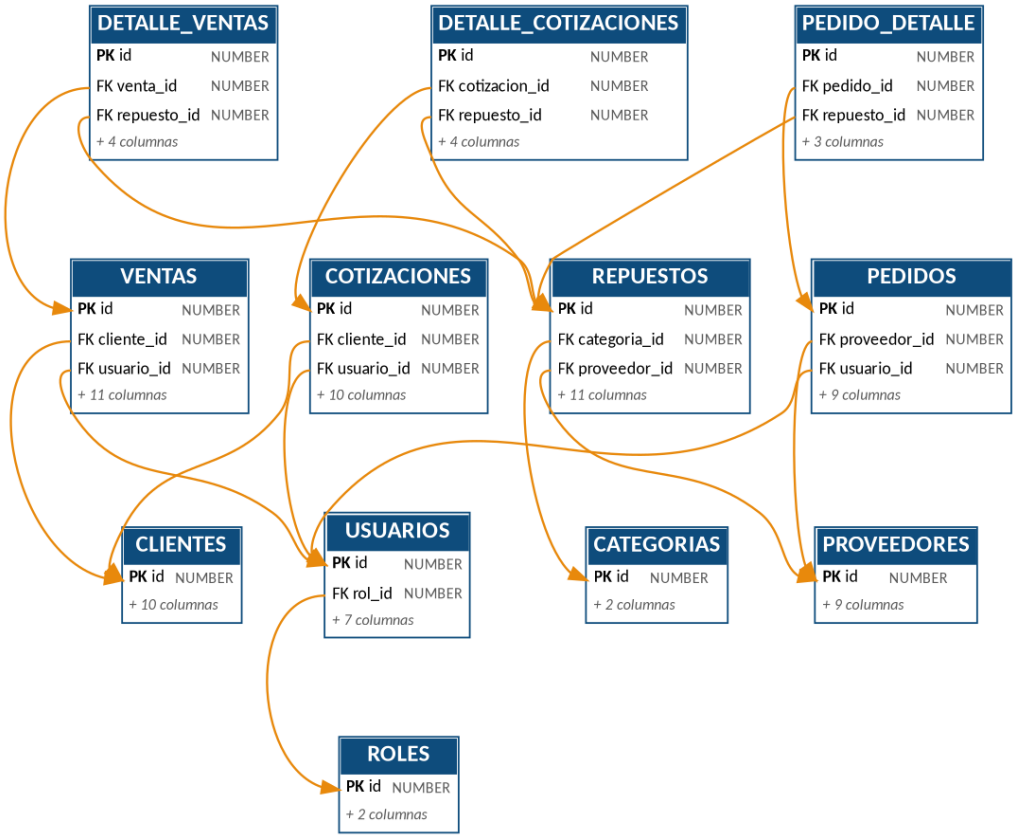


Figura 1. Módulo comercial: usuarios, clientes, inventario, ventas, cotizaciones y pedidos

2.2 Vehículos y compatibilidad

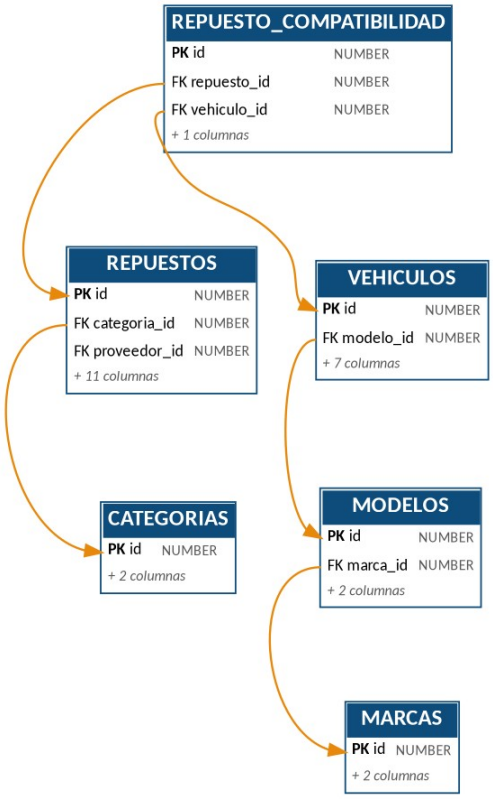


Figura 2. Vehículos y compatibilidad (con MARCAS y MODELOS)

Un repuesto es compatible con muchos vehículos y un vehículo acepta muchos repuestos: la tabla REPUESTO_COMPATIBILIDAD resuelve la relación N:M. Desde el script 15, un vehículo apunta a un modelo y cada modelo a una marca.

2.3 Asistente, auditoría y seguridad

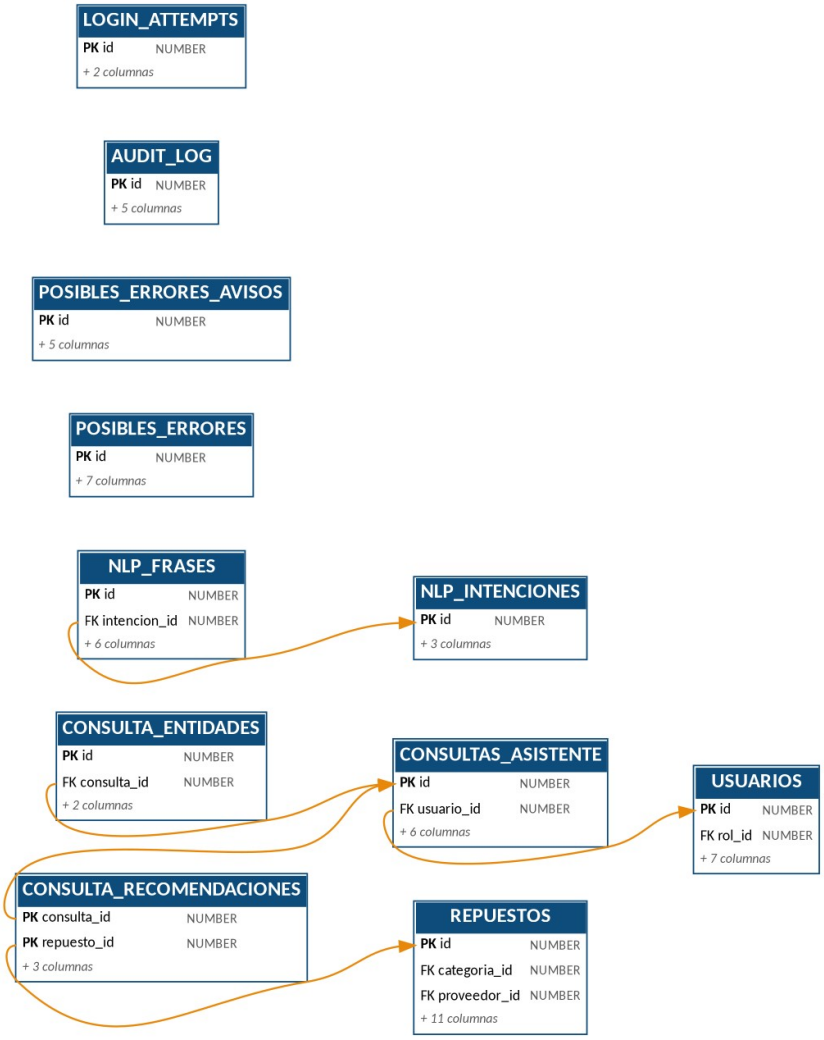


Figura 3. Asistente de lenguaje natural, auditoría y seguridad

NLP_INTENCIONES y NLP_FRASES guardan el corpus de entrenamiento; POSIBLES_ERRORES y POSIBLES_ERRORES_AVISOS son el catálogo de errores de digitación; CONSULTAS_ASISTENTE guarda cada consulta y, desde el script 15, sus entidades y recomendaciones quedan en tablas hijas. AUDIT_LOG y LOGIN_ATTEMPTS no tienen claves foráneas a propósito: deben conservar el registro aunque se borre el usuario.

3. Diccionario de datos

Para cada tabla: nombre de columna, tipo Oracle, si admite nulos (NN = NOT NULL), tipo de clave y observaciones (valor por defecto, clave foránea, restricciones de unicidad y CHECK).

ROLES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(50)	Sí		Único
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: nombre

USUARIOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(100)	Sí		
email	VARCHAR2(150)	Sí		Único
password	VARCHAR2(255)	Sí		
rol_id	NUMBER	Sí	FK	→ roles(id)
activo	NUMBER(1)			Def: 1
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
updated_at	TIMESTAMP			
deleted_at	TIMESTAMP			

Unicidad: email

PROVEEDORES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(150)	Sí		
contacto	VARCHAR2(100)			
telefono	VARCHAR2(30)			
email	VARCHAR2(150)			
direccion	VARCHAR2(4000)			
activo	NUMBER(1)			Def: 1
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Columna	Tipo	NN	Clave	Observaciones
updated_at	TIMESTAMP			
deleted_at	TIMESTAMP			

CATEGORIAS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(100)	Sí		Único
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: nombre

REPUESTOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
codigo	VARCHAR2(50)			Único
nombre	VARCHAR2(200)	Sí		
descripcion	VARCHAR2(4000)			
categoria_id	NUMBER		FK	→ categorias(id)
proveedor_id	NUMBER		FK	→ proveedores(id)
precio_compra	NUMBER(12,2)	Sí		Def: 0
precio_venta	NUMBER(12,2)	Sí		Def: 0
stock	NUMBER(10)	Sí		Def: 0
stock_minimo	NUMBER(10)	Sí		Def: 5
activo	NUMBER(1)			Def: 1
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
updated_at	TIMESTAMP			
deleted_at	TIMESTAMP			

Unicidad: codigo · CHECK: CONSTRAINT ck_repuestos_stock CHECK (stock >= 0) | CONSTRAINT ck_repuestos_precios CHECK (precio_venta >= 0 AND precio_compra >= 0)

CLIENTES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(150)	Sí		

Columna	Tipo	NN	Clave	Observaciones
identificacion	VARCHAR2(30)			Único
tipo_doc	VARCHAR2(20)	Sí		Def: 'CC'
telefono	VARCHAR2(30)			
email	VARCHAR2(150)			
direccion	VARCHAR2(4000)			
activo	NUMBER(1)			Def: 1
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
updated_at	TIMESTAMP			
deleted_at	TIMESTAMP			

Unicidad: identificacion · CHECK: CONSTRAINT ck_clientes_tipo_doc CHECK (tipo_doc IN ('CC','NIT','CE','Pasaporte'))

VENTAS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
numero	VARCHAR2(20)	Sí		Único
cliente_id	NUMBER		FK	→ clientes(id)
usuario_id	NUMBER	Sí	FK	→ usuarios(id)
subtotal	NUMBER(14,2)	Sí		Def: 0
descuento	NUMBER(14,2)	Sí		Def: 0
impuesto	NUMBER(14,2)	Sí		Def: 0
total	NUMBER(14,2)	Sí		Def: 0
metodo_pago	VARCHAR2(20)	Sí		Def: 'efectivo'
estado	VARCHAR2(20)	Sí		Def: 'completada'
notas	VARCHAR2(4000)			
version	NUMBER			Def: 1
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
updated_at	TIMESTAMP			

Unicidad: numero · CHECK: CONSTRAINT ck_ventas_metodo_pago CHECK (metodo_pago IN ('efectivo','tarjeta','transferencia','credito')) | CONSTRAINT ck_ventas_estado CHECK (estado IN ('completada','anulada','pendiente')) | CONSTRAINT ck_ventas_total CHECK (total >= 0)

DETALLE_VENTAS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
venta_id	NUMBER	Sí	FK	→ ventas(id)
repuesto_id	NUMBER	Sí	FK	→ repuestos(id)
cantidad	NUMBER(10)	Sí		Def: 1
precio_unit	NUMBER(12,2)	Sí		
descuento	NUMBER(12,2)	Sí		Def: 0
subtotal	NUMBER(12,2)	Sí		

CHECK: CONSTRAINT ck_detven_cantidad CHECK (cantidad > 0)

COTIZACIONES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
numero	VARCHAR2(20)	Sí		Único
cliente_id	NUMBER		FK	→ clientes(id)
usuario_id	NUMBER	Sí	FK	→ usuarios(id)
subtotal	NUMBER(14,2)	Sí		Def: 0
descuento	NUMBER(14,2)	Sí		Def: 0
impuesto	NUMBER(14,2)	Sí		Def: 0
total	NUMBER(14,2)	Sí		Def: 0
estado	VARCHAR2(20)	Sí		Def: 'borrador'
vence_en	DATE			
notas	VARCHAR2(4000)			
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
updated_at	TIMESTAMP			

Unicidad: numero · CHECK: CONSTRAINT ck_cotiz_estado CHECK (estado IN ('borrador','enviada','aceptada','rechazada','vencida'))

DETALLE_COTIZACIONES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
cotizacion_id	NUMBER	Sí	FK	→ cotizaciones(id)
repuesto_id	NUMBER	Sí	FK	→ repuestos(id)

Columna	Tipo	NN	Clave	Observaciones
cantidad	NUMBER(10)	Sí		Def: 1
precio_unit	NUMBER(12,2)	Sí		
descuento	NUMBER(12,2)	Sí		Def: 0
subtotal	NUMBER(12,2)	Sí		

VEHICULOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
marca	VARCHAR2(80)	Sí		
modelo	VARCHAR2(80)	Sí		
anio_desde	NUMBER(4)	Sí		
anio_hasta	NUMBER(4)	Sí		
motor	VARCHAR2(50)			
combustible	VARCHAR2(20)	Sí		Def: 'gasolina'
creado_en	TIMESTAMP			Def: SYSTIMESTAMP
modelo_id	NUMBER		FK	→ modelos(id)

Unicidad: marca,modelo,anio_desde,anio_hasta,motor,combustible · CHECK: CONSTRAINT ck_vehiculos_combustible CHECK (combustible IN ('gasolina','diesel','hibrido','electrico','gas')) | CONSTRAINT ck_vehiculos_anios CHECK (anio_hasta >= anio_desde)

REPUESTO_COMPATIBILIDAD

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
repuesto_id	NUMBER	Sí	FK	→ repuestos(id)
vehiculo_id	NUMBER	Sí	FK	→ vehiculos(id)
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: repuesto_id,vehiculo_id

CONSULTAS_ASISTENTE

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
usuario_id	NUMBER		FK	→ usuarios(id)
texto_consulta	VARCHAR2(4000)	Sí		
intencion_detectada	VARCHAR2(50)			

Columna	Tipo	NN	Clave	Observaciones
entidades_json	JSON			
respuesta_json	JSON			
util	NUMBER(1)			Def: NULL
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

AUDIT_LOG

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
usuario_id	NUMBER			
accion	VARCHAR2(100)	Sí		
detalle	VARCHAR2(4000)			
ip	VARCHAR2(45)			
created_at	TIMESTAMP			Def: SYSTIMESTAMP

LOGIN_ATTEMPTS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
ip	VARCHAR2(45)	Sí		
created_at	TIMESTAMP			Def: SYSTIMESTAMP

PEDIDOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
numero	VARCHAR2(20)	Sí		Único
proveedor_id	NUMBER	Sí	FK	→ proveedores(id)
usuario_id	NUMBER	Sí	FK	→ usuarios(id)
estado	VARCHAR2(20)	Sí		Def: 'solicitado'
subtotal	NUMBER(14,2)	Sí		Def: 0
total	NUMBER(14,2)	Sí		Def: 0
fecha_esperada	DATE			
notas	VARCHAR2(4000)			
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Columna	Tipo	NN	Clave	Observaciones
updated_at	TIMESTAMP			
recibido_en	TIMESTAMP			

Unicidad: numero · CHECK: CONSTRAINT ck_pedidos_estado CHECK (estado IN ('solicitado','en_transito','recibido','cancelado')) | CONSTRAINT ck_pedidos_total CHECK (total >= 0 AND subtotal >= 0)

PEDIDO_DETALLE

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
pedido_id	NUMBER	Sí	FK	→ pedidos(id)
repuesto_id	NUMBER	Sí	FK	→ repuestos(id)
cantidad	NUMBER(10)	Sí		Def: 1
precio_unit	NUMBER(12,2)	Sí		
subtotal	NUMBER(12,2)	Sí		

CHECK: CONSTRAINT ck_peddet_cantidad CHECK (cantidad > 0) | CONSTRAINT ck_peddet_precio CHECK (precio_unit >= 0)

NLP_INTENCIONES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(50)	Sí		Único
descripcion	VARCHAR2(200)	Sí		
respuesta	VARCHAR2(30)	Sí		

Unicidad: nombre

NLP_FRASES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
texto	VARCHAR2(1000)	Sí		
intencion_id	NUMBER	Sí	FK	→ nlp_intenciones(id)
conjunto	VARCHAR2(15)	Sí		
n_caracteres	NUMBER(4)	Sí		
n_palabras	NUMBER(3)	Sí		
fuente	VARCHAR2(100)	Sí		
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

CHECK: CONSTRAINT ck_nlp_frases_conjunto CHECK (conjunto IN ('entrenamiento', 'holdout'))

POSIBLES_ERRORES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
tipo	VARCHAR2(10)	Sí		
error	VARCHAR2(100)	Sí		
correcto	VARCHAR2(100)	Sí		
nivel	NUMBER(1)	Sí		
origen	VARCHAR2(10)	Sí		Def: 'auto'
nota	VARCHAR2(100)			
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: tipo,error,correcto · CHECK: CONSTRAINT ck_pe_tipo CHECK (tipo IN ('marca', 'modelo', 'categoria')) | CONSTRAINT ck_pe_nivel CHECK (nivel BETWEEN 1 AND 4) | CONSTRAINT ck_pe_origen CHECK (origen IN ('auto', 'manual'))

POSIBLES_ERRORES_AVISOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
tipo	VARCHAR2(10)	Sí		
error	VARCHAR2(100)	Sí		
correcto	VARCHAR2(100)	Sí		
nivel	NUMBER(1)	Sí		
motivo	VARCHAR2(100)	Sí		

CHECK: CONSTRAINT ck_pea_tipo CHECK (tipo IN ('marca', 'modelo', 'categoria'))

MARCAS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
nombre	VARCHAR2(80)	Sí		Único
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: nombre

MODELOS

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
marca_id	NUMBER	Sí	FK	→ marcas(id)
nombre	VARCHAR2(80)	Sí		

Columna	Tipo	NN	Clave	Observaciones
creado_en	TIMESTAMP			Def: SYSTIMESTAMP

Unicidad: marca_id,nombre

CONSULTA_ENTIDADES

Columna	Tipo	NN	Clave	Observaciones
id	NUMBER	Sí	PK	Def: IDENTITY
consulta_id	NUMBER	Sí	FK	→ consultas_asistente(id)
tipo	VARCHAR2(30)	Sí		
valor	VARCHAR2(200)	Sí		

CHECK: CHECK (tipo IN ('marca','modelo','anio','categoria','repuesto','cantidad'))

CONSULTA_RECOMENDACIONES

Columna	Tipo	NN	Clave	Observaciones
consulta_id	NUMBER	Sí	PK	→ consultas_asistente(id)
repuesto_id	NUMBER	Sí	PK	→ repuestos(id)
posicion	NUMBER(3)	Sí		
es_sustituto	NUMBER(1)	Sí		Def: 0
puntaje	NUMBER(5,2)			

CHECK: CHECK (es_sustituto IN (0,1)) | CHECK (posicion > 0)

4. Normalización

Se revisó cada tabla contra las tres primeras formas normales y la forma normal de Boyce-Codd (FNBC). La revisión se hizo sobre el esquema real; lo que ya cumplía se deja tal cual y lo que no, se corrige con el script 15.

4.1 Primera forma normal (1FN)

- Todas las columnas son atómicas: no hay listas ni columnas repetidas del tipo `telefono1`, `telefono2`.
- Todas las tablas tienen clave primaria (columna `id` con identity), salvo CONSULTA_RECOMENDACIONES, cuya clave es compuesta (consulta_id, repuesto_id).
- Incumplimiento detectado: CONSULTAS_ASISTENTE guardaba las entidades extraídas (`entidades_json`) y la respuesta con los repuestos recomendados (`respuesta_json`) como documentos JSON dentro de una sola celda, es decir, varios valores por columna. Se crearon CONSULTA_ENTIDADES y CONSULTA_RECOMENDACIONES para guardar un valor por fila.

4.2 Segunda forma normal (2FN)

Como casi todas las tablas usan una clave simple (`id`), no pueden tener dependencias parciales. Las únicas con clave compuesta son REPUESTO_COMPATIBILIDAD y CONSULTA_RECOMENDACIONES; en ambas los atributos dependen de la clave completa (la posición, el puntaje y el indicador de sustituto describen el par consulta–repuesto). **Cumple.**

4.3 Tercera forma normal (3FN)

Dependencias transitivas encontradas y su tratamiento:

Tabla.columna	Problema	Solución
VEHICULOS.marca / modelo	El texto de la marca depende del modelo, y el modelo de su marca: se repetía en cada vehículo (modelo → marca).	Tablas MARCAS y MODELOS; VEHICULOS.modelo_id (FK). Fase 1: se conservan las columnas de texto y un trigger llena modelo_id. Fase 2: eliminarlas.
VENTAS.subtotal / total, COTIZACIONES.subtotal / total, PEDIDOS.subtotal / total	Valores derivados de la suma de las líneas: pueden quedar desincronizados.	Vista `vw_totales_documento` calcula y concilia (columna `diferencia`). Fase 2: dejar de leer las columnas guardadas.
DETALLE_*.subtotal	Derivado: cantidad × precio_unit − descuento.	Se mantiene por rendimiento y se valida con la vista; es una desnormalización controlada.

El resto de las tablas ya está en 3FN: ROLES, CATEGORIAS, PROVEEDORES, CLIENTES, USUARIOS (el rol se guarda como FK), REPUESTOS (categoría y proveedor como FK) y las tablas NLP.

4.4 Forma normal de Boyce-Codd (FNBC)

En todas las tablas los determinantes son claves candidatas: la clave primaria `id` más las restricciones de unicidad (`usuarios.email`, `repuestos.codigo`, `ventas.numero`, `cotizaciones.numero`, `pedidos.numero`, `marcas.nombre`, `modelos(marca_id, nombre)`). No se encontró ninguna dependencia funcional cuyo determinante no sea clave candidata, por lo que las tablas cumplen FNBC.

4.5 Lo que queda para la fase 2

Pendiente: cambios en el backend (PHP)

La fase 1 es **compatible hacia atrás**: no rompe el backend actual porque deja las columnas viejas. La fase 2 exige tocar código:

1. `VehiculoModel` debe leer y escribir `modelo_id` (usando `vw_vehiculos`) en lugar de `marca` y `modelo` como texto.
2. Las rutas de ventas y cotizaciones deben dejar de depender de `subtotal` y `total` guardados y usar `vw_totales_documento`.
3. Luego se eliminan `vehiculos.marca`, `vehiculos.modelo` y las columnas derivadas. Esto se hará en un sprint posterior, con pruebas.

5. Vistas y verificaciones

Vista	Propósito
vw_totales_documento	Une ventas, cotizaciones y pedidos: suma de detalles (`subtotal_detalle`), descuento e impuesto del encabezado, `total_calculado`, `total_guardado` y `diferencia`. El IVA aún no se puede derivar de las líneas.
vw_vehiculos	Devuelve vehículo con marca y modelo normalizados (joins con MODELOS y MARCAS).

Verificaciones al final del script 15: vehículos sin `modelo_id` = 0 y documentos con `diferencia <> 0` = 0. Si alguna no da cero, el script lo informa para revisar los datos antes de continuar.